

CSC1003 Tutorial 3

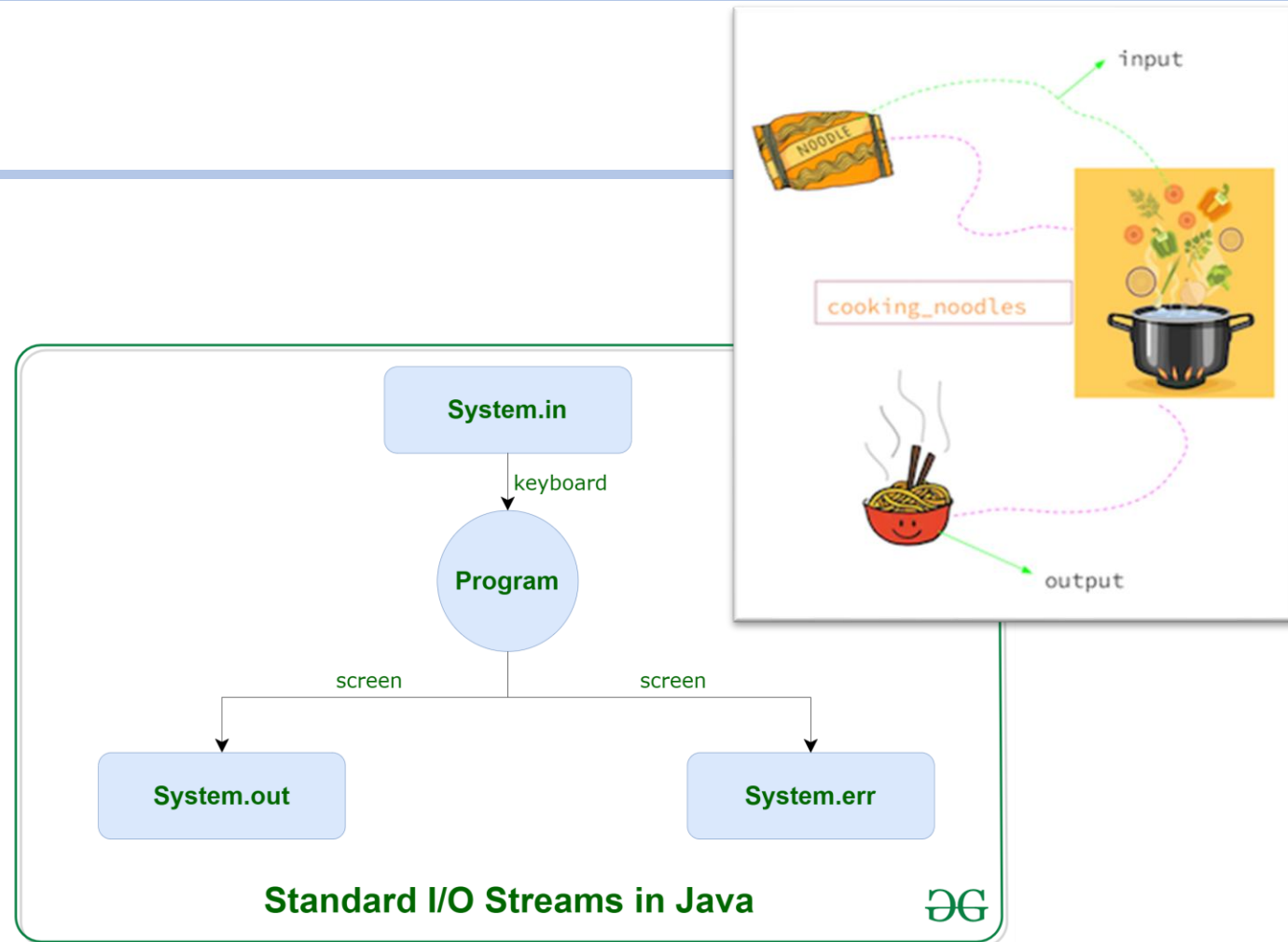
Input / Output

Overview

- I/O Concept
- I/O Operation in Java
 - Command line arguments
 - Standard input stream
 - Standard output stream
- Drawing in Java using StdDraw (Optional)
- Coding Style

I/O Concept

- **Input:** How your program takes in data
- **Output:** How your program sends out data
- Typical sources and destinations:
 - **System.in, System.out, System.err**
 - Files
 - Pipes
 - Network Connections
 - In-memory Buffers (arrays, etc.)



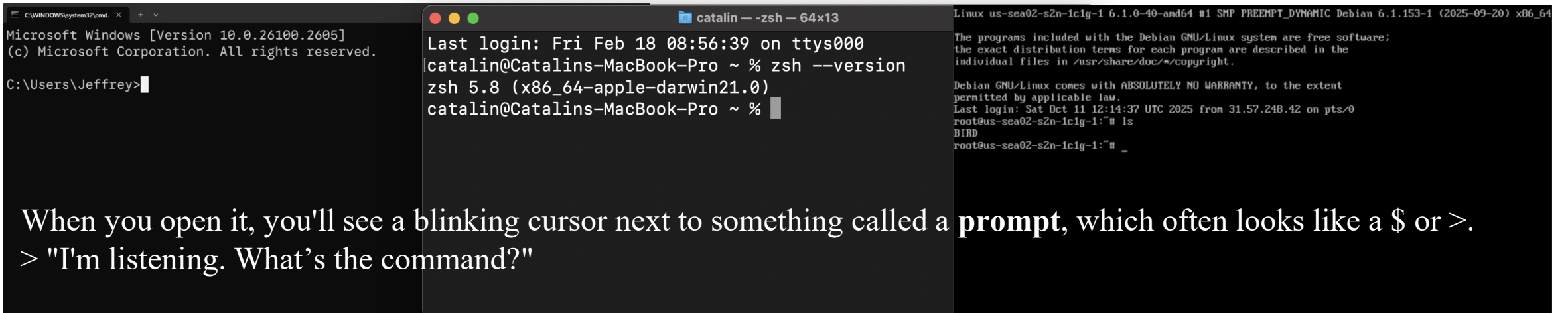
Command Line Arguments

0. What is Command Line Interface (CLI)?

Opposite of GUI (Graphical User Interface) that you use everyday.

GUI: You see and click;

Command Line: You type and execute.



The image displays three terminal windows side-by-side, illustrating the Command Line Interface (CLI) across different operating systems:

- Windows:** The terminal window shows the Microsoft Windows command prompt. The title bar indicates the path `C:\WINDOWS\system32\cmd.exe`. The text inside reads: `Microsoft Windows [Version 10.0.26100.2605]
(c) Microsoft Corporation. All rights reserved.
C:\Users\Jeffrey>`
- macOS:** The terminal window is titled `catalin — zsh — 64x13`. It shows the login history and the version of the `zsh` shell: `Last login: Fri Feb 18 08:56:39 on ttys000
catalin@Catalins-MacBook-Pro ~ % zsh --version
zsh 5.8 (x86_64-apple-darwin21.0)
catalin@Catalins-MacBook-Pro ~ %`
- Linux:** The terminal window shows the Debian GNU/Linux login screen. The title bar indicates the path `Linux us-sea02-s2n-1c1g-1 6.1.0-40-and64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.153-1 (2025-09-20) x86_64`. The text inside reads: `The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sat Oct 11 12:14:37 UTC 2025 from 31.57.248.42 on pts/0
root@us-sea02-s2n-1c1g-1:~# ls
BIRD
root@us-sea02-s2n-1c1g-1:~#`

When you open it, you'll see a blinking cursor next to something called a **prompt**, which often looks like a `$` or `>`.
> "I'm listening. What's the command?"

Command Line Arguments

1. What are command line arguments?

A string array argument that is provided to the [main\(\)](#) method, the typical entry point for a Java class.

The JVM (Java Virtual Machine) passes the command line argument **the moment you run your program**, through the `String[] args` parameter of [main\(\)](#), so you can fetch these arguments by accessing this parameter.

```
Class SimpleCommandLinePgm {  
    public static void main(String[] args) {  
        System.out.println("This is a simple command line program!");  
        System.out.println("Your Command Line arguments are:"); //  
        for (String str: args) {  
            System.out.println(str);  
        }  
    }  
}
```

Command Line Arguments

2. How can you use them?

You need to pass the arguments as **space-separated values**.

You can pass Strings and primitive data types (int, double, etc.) as command-line arguments, but they need to be parsed (*Integer.parseInt(); Double.parseDouble(); etc.*)

```
java SimpleCommandLinePgm one two three four
```

← executed in CLI

```
This is a simple command line program!
```

```
Your Command Line arguments are:
```

```
one
```

```
two
```

```
three
```

```
four
```

Command Line Arguments

3. Limitations

- ~~The length of command line arguments is limited.~~
- **Non-interactive** (might be a BIG problem when you want to interact with users or deal with unexpected inputs).

Stream

- Stream:
 - (of water) flows from a source (like a spring) to a destination (like a lake).
take water out of the stream or pour water into it.
 - (in Java) a sequence of data that **flows** from a source to a destination.
-
- Input Stream:
 - For reading data. It's like putting a bucket into the river to collect water.
Source (keyboard, file) → your program.
 - Output Stream:
 - For writing data. It's like pouring a bucket of water into the river.
Your program → destination (screen, file).

Standard Input Stream

1. What is standard input stream?

The standard input (stdin) can be represented by `System.in` in Java.
The `System.in` is an instance of the `InputStream` class.

To read any data from a keyboard (`System.in`), we can use a **Scanner** class.

Standard Input Stream

2. How to create a Scanner

```
import java.util.Scanner;  
public class NextCharScanner{  
  
    // Java Scanner char input example  
    public static void main(String[] args) {  
        System.out.println("Provide the Java Scanner char input: ");  
        Scanner charScanner = new Scanner(System.in);  
        charScanner.useDelimiter("");  
        while (charScanner.hasNext()) {  
            char name = charScanner.next().charAt(0);  
            if (name == '\n') {  
                return;  
            }  
        }  
    }  
}
```

Import the package, *java.util.Scanner*.

Create a scanner object
Instead of *System.in*, what else can you put here?

During input (via keyboard), you need to press ENTER, otherwise it will be blocked here.
That behavior is called Blocking, i.e. entering a waiting state.

Standard Input Stream

You may lookup the *Scanner* document.

<<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Scanner.html>>

Constructors	
Constructor	Description
<code>Scanner(File source)</code>	Constructs a new Scanner that produces values scanned from the specified file.
<code>Scanner(File source, String charsetName)</code>	Constructs a new Scanner that produces values scanned from the specified file.
<code>Scanner(File source, Charset charset)</code>	Constructs a new Scanner that produces values scanned from the specified file.
<code>Scanner(InputStream source)</code>	Constructs a new Scanner that produces values scanned from the specified input stream.
<code>Scanner(InputStream source, String charsetName)</code>	Constructs a new Scanner that produces values scanned from the specified input stream.
<code>Scanner(InputStream source, Charset charset)</code>	Constructs a new Scanner that produces values scanned from the specified input stream.
<code>Scanner(Readable source)</code>	Constructs a new Scanner that produces values scanned from the specified source.
<code>Scanner(String source)</code>	Constructs a new Scanner that produces values scanned from the specified string.
<code>Scanner(ReadableByteChannel source)</code>	Constructs a new Scanner that produces values scanned from the specified channel.
<code>Scanner(ReadableByteChannel source, String charsetName)</code>	Constructs a new Scanner that produces values scanned from the specified channel.
<code>Scanner(ReadableByteChannel source, Charset charset)</code>	Constructs a new Scanner that produces values scanned from the specified channel.
<code>Scanner(Path source)</code>	Constructs a new Scanner that produces values scanned from the specified file.
<code>Scanner(Path source, String charsetName)</code>	Constructs a new Scanner that produces values scanned from the specified file.
<code>Scanner(Path source, Charset charset)</code>	Constructs a new Scanner that produces values scanned from the specified file.

```
//Declare File object
File file = new File("/home/javatpoint/Desktop/ABHISHEK/AngularJS/Index/abc.txt");
//Initialize the scanner
Scanner scan = new Scanner(file);
// iterate through the file line by line
while(scan.hasNextLine()){
    //Print the contents of a file by line
    System.out.println(scan.next());
}
```

```
String s = "Facebook.com \n JavaTpoint.com 22 60.0";
//Create a new scanner with the specified String Object
Scanner scanner = new Scanner(s);
//Find the next token and print it
System.out.print("Token Value1 " + scanner.next());
System.out.print("\nToken value2: " + scanner.next());
```

Standard Input Stream

Read input using a series of “next()” function.

String	next()	Finds and returns the next complete token from this scanner.
String	next(String pattern)	Returns the next token if it matches the pattern constructed from the specified string.
String	next(Pattern pattern)	Returns the next token if it matches the specified pattern.
BigDecimal	nextBigDecimal()	Scans the next token of the input as a <code>BigDecimal</code> .
BigInteger	nextBigInteger()	Scans the next token of the input as a <code>BigInteger</code> .
BigInteger	nextBigInteger(int radix)	Scans the next token of the input as a <code>BigInteger</code> .
boolean	nextBoolean()	Scans the next token of the input into a boolean value and returns that value.
byte	nextByte()	Scans the next token of the input as a <code>byte</code> .
byte	nextByte(int radix)	Scans the next token of the input as a <code>byte</code> .
double	nextDouble()	Scans the next token of the input as a <code>double</code> .
float	nextFloat()	Scans the next token of the input as a <code>float</code> .
int	nextInt()	Scans the next token of the input as an <code>int</code> .
int	nextInt(int radix)	Scans the next token of the input as an <code>int</code> .
String	nextLine()	Advances this scanner past the current line and returns the input that was skipped.
long	nextLong()	Scans the next token of the input as a <code>long</code> .
long	nextLong(int radix)	Scans the next token of the input as a <code>long</code> .
short	nextShort()	Scans the next token of the input as a <code>short</code> .
short	nextShort(int radix)	Scans the next token of the input as a <code>short</code> .

```
import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        Scanner myObj = new Scanner(System.in);

        System.out.println("Enter name, age and salary:");

        // String input
        String name = myObj.nextLine();

        // Numerical input
        int age = myObj.nextInt();
        double salary = myObj.nextDouble();

        // Output input by user
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Salary: " + salary);
    }
}
```

What happens if I put a double here?

Standard Input Stream

Check input using a series of “hasNext()” function.

You can use them with a while loop to continuously read inputs and stop when you want.

boolean	<code>hasNext()</code>	Returns true if this scanner has another token in its input.
boolean	<code>hasNext(String pattern)</code>	Returns true if the next token matches the pattern constructed from the specified string.
boolean	<code>hasNext(Pattern pattern)</code>	Returns true if the next complete token matches the specified pattern.
boolean	<code>hasNextBigDecimal()</code>	Returns true if the next token in this scanner's input can be interpreted as a <code>BigDecimal</code> using the <code>nextBigDecimal()</code> method.
boolean	<code>hasNextBigInteger()</code>	Returns true if the next token in this scanner's input can be interpreted as a <code>BigInteger</code> in the default radix using the <code>nextBigInteger()</code> method.
boolean	<code>hasNextBigInteger(int radix)</code>	Returns true if the next token in this scanner's input can be interpreted as a <code>BigInteger</code> in the specified radix using the <code>nextBigInteger()</code> method.
boolean	<code>hasNextBoolean()</code>	Returns true if the next token in this scanner's input can be interpreted as a boolean value using a case insensitive pattern created from the string "true false".
boolean	<code>hasNextByte()</code>	Returns true if the next token in this scanner's input can be interpreted as a byte value in the default radix using the <code>nextByte()</code> method.
boolean	<code>hasNextByte(int radix)</code>	Returns true if the next token in this scanner's input can be interpreted as a byte value in the specified radix using the <code>nextByte()</code> method.
boolean	<code>hasNextDouble()</code>	Returns true if the next token in this scanner's input can be interpreted as a double value using the <code>nextDouble()</code> method.
boolean	<code>hasNextFloat()</code>	Returns true if the next token in this scanner's input can be interpreted as a float value using the <code>nextFloat()</code> method.
boolean	<code>hasNextInt()</code>	Returns true if the next token in this scanner's input can be interpreted as an int value in the default radix using the <code>nextInt()</code> method.
boolean	<code>hasNextInt(int radix)</code>	Returns true if the next token in this scanner's input can be interpreted as an int value in the specified radix using the <code>nextInt()</code> method.
boolean	<code>hasNextLine()</code>	Returns true if there is another line in the input of this scanner.
boolean	<code>hasNextLong()</code>	Returns true if the next token in this scanner's input can be interpreted as a long value in the default radix using the <code>nextLong()</code> method.
boolean	<code>hasNextLong(int radix)</code>	Returns true if the next token in this scanner's input can be interpreted as a long value in the specified radix using the <code>nextLong()</code> method.
boolean	<code>hasNextShort()</code>	Returns true if the next token in this scanner's input can be interpreted as a short value in the default radix using the <code>nextShort()</code> method.
boolean	<code>hasNextShort(int radix)</code>	Returns true if the next token in this scanner's input can be interpreted as a short value in the specified radix using the <code>nextShort()</code> method.

Standard Input Stream

```
public class main {  
  
    public static void main(String[] args){  
  
        Scanner Input = new Scanner(System.in);  
        int count = 0;  
        int temp;  
        int sum = 0;  
  
        while(Input.hasNextInt()){  
  
            count++;  
  
            temp = Input.nextInt();  
            sum += temp;  
            System.out.println(temp);  
            System.out.println(count);  
  
        } // End of while  
  
        double average = sum / count;  
        System.out.println("The average is: " + average);  
  
    } // End of method main  
  
}
```

← You need to type a delimiter (NaN) to break the while loop.

Standard Output Stream

- *System* is a class in Java;
- *out* is one of the fields of *System* class → the standard output stream.
- `println` is a method.

out

```
public static final PrintStream out
```

The "standard" output stream. This stream is already open and ready to accept output data. Typically this stream corresponds to display output or another output destination specified by the host environment or user.

For simple stand-alone Java applications, a typical way to write a line of output data is:

```
System.out.println(data)
```

See the `println` methods in class `PrintStream`.

See Also:

```
PrintStream.println(), PrintStream.println(boolean), PrintStream.println(char), PrintStream.println(char[]), PrintStream.println(double), PrintStream.println(float), PrintStream.println(int),  
PrintStream.println(long), PrintStream.println(java.lang.Object), PrintStream.println(java.lang.String)
```

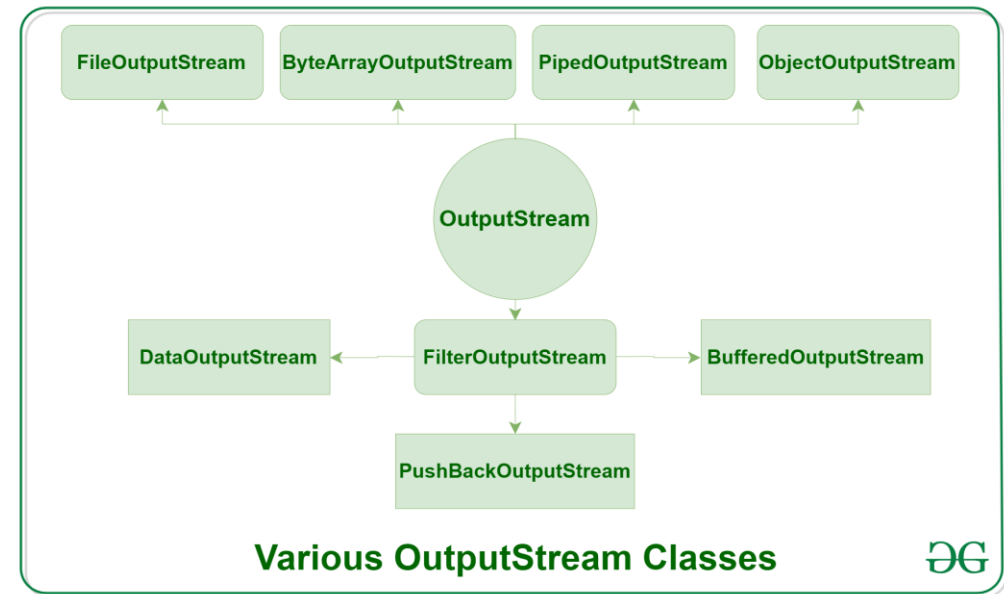
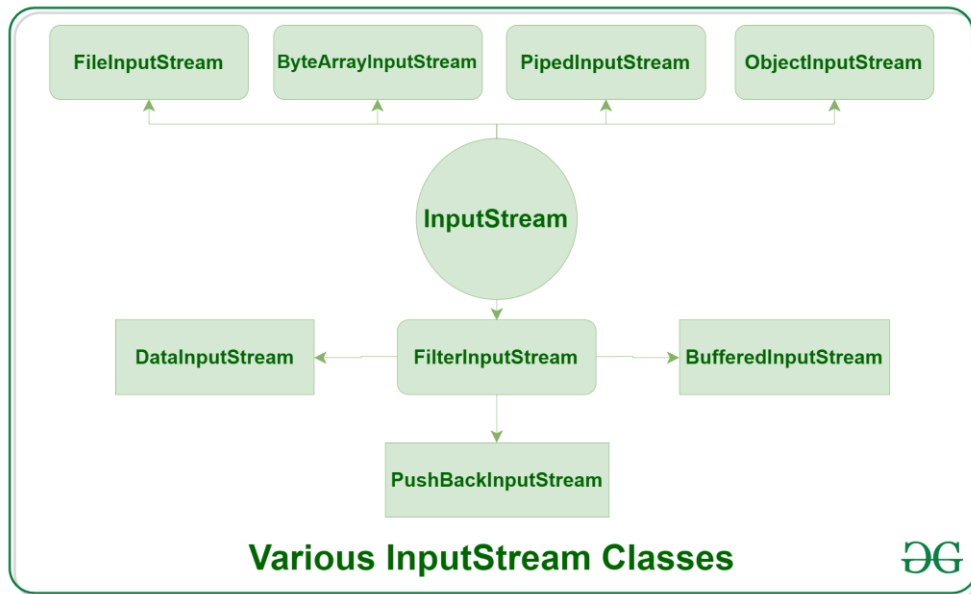
Other methods of out? (Something you can play with.)

<https://docs.oracle.com/javase/8/docs/api/java/lang/System.html#out>

```
long n = 461012;  
System.out.format("%d%n", n);      // --> "461012"  
System.out.format("%08d%n", n);   // --> "00461012"  
System.out.format("%+8d%n", n);   // --> " +461012"  
System.out.format("%,8d%n", n);   // --> " 461,012"  
System.out.format("%+,8d%n%n", n); // --> "+461,012"
```

Stream

- Java includes a rich set of I/O classes, mostly in the `java.io` package.



- Scanner;
- BufferedReader;
-

StdDraw (*Optional*)

- You may want to output pictures instead of text in some cases.
- Some well-known tool:
In this course we introduce a StdDraw library. (A good chance to learn to discover a library.)
<<https://introcs.cs.princeton.edu/java/stdlib/javadoc/StdDraw.html>>
- To use that library, you might choose several ways below.

Using the standard libraries. The file `stdlib.jar` bundles together all of our standard libraries into one file. To access the libraries, you must add `stdlib.jar` to your *Java classpath*. There are many ways to do so (and many opportunities to make mistakes). Here are the two recommended ways:

OS X / Linux / Git Bash

```
% javac -classpath .:stdlib.jar MyProgram.java
% java -classpath .:stdlib.jar MyProgram
```

Windows Command Prompt

```
% javac -classpath .;stdlib.jar MyProgram.java
% java -classpath .;stdlib.jar MyProgram
```

```
% jar xf stdlib.jar
```

```
% javac MyProgram.java
% java MyProgram
```

StdDraw (*Optional*)

```
package lab3;
import java.awt.Color;

import stdlib.*;

public class HappyFace
{
    public static void main(String[] args)
    {
        // Set up the canvas and X/Y scales
        StdDraw.setXscale(0, 100); // x scale from 0 to 100
        StdDraw.setYscale(0, 100); // y scale from 0 to 100

        // Face as an yellow circle
        StdDraw.setPenColor(Color.YELLOW);
        StdDraw.filledCircle(50, 50, 30);

        // Mouth as an arc
        StdDraw.setPenColor(Color.BLACK);
        StdDraw.setPenRadius(0.01); // somewhat thick
        StdDraw.arc(50, 50, 20, 210, 330);

        // **
        // ** WRITE YOUR CODE HERE
        // **

    }
}
```



Coding Style

```
1 import java.util.Scanner;public class tmp {
2     public static void main(String[] args) {
3         final Scanner scan=new Scanner(System.in);
4 final int x= scan.nextInt();
5         final int y =scan.nextInt();final int z=x+y;
6     System.out.println(x+" + "+y +" = "+ z);
7     }
8 }
```



- **Shorter?**
- **What the hell is it talking about!**

```
1 import java.util.Scanner;
2
3 public class tmp {
4     public static void main(String[] args) {
5         final Scanner scan = new Scanner(System.in);
6
7         final int x = scan.nextInt();
8         final int y = scan.nextInt();
9         final int z = x + y;
10
11         System.out.println(x + " + " + y + " = " + z);
12     }
13 }
```

- **Much easier to read and maintain.**

Coding Style

```
1 import java.util.Scanner;public class tmp {
2     public static void main(String[] args) {
3         final Scanner scan=new Scanner(System.in);
4 final int x= scan.nextInt();
5         final int y =scan.nextInt();final int z=x+y;
6     System.out.println(x+" + "+y +" = "+ z);
7     }
8 }
```

```
1 import java.util.Scanner;
2
3 public class tmp {
4     public static void main(String[] args) {
5         final Scanner scan = new Scanner(System.in);
6
7         final int x = scan.nextInt();
8         final int y = scan.nextInt();
9         final int z = x + y;
10
11     System.out.println(x + " + " + y + " = " + z);
12 }
13 }
```

Both of them doesn't close Scanner.

→ Resource Leak

(when you open a Scanner, you're requesting OS for a resource, and the total resource is limited.)

Coding Style

How to format your source file?

- Form a habit to write clean code.

Reading: Google Java Style Code

<https://google.github.io/styleguide/javaguide.html>

- Use Code-Formatting feature of VS Code (other IDEs also includes such feature):

Windows: Alt + Shift + F

MacOS: Option + Shift + F

Q & A